

Procesamiento Distribuido

Autor

Martín Vladimir Alonso Sierra Galvis

Gestión de Datos, Maestría en Ciencia de Datos
Pontificia Universidad Javeriana Cali

Versión 2.0
Santiago de Cali, marzo de 2023

Tabla de contenido

Conceptos de Procesamiento Distribuido	2
¿Qué es el Procesamiento de Datos?	2
¿Qué es el Procesamiento Distribuido de Datos?	2
Modelos de Procesamiento Distribuido	3
Modelo MapReduce	3
Modelo Directed Acyclic Graph, DAG	5
Tipos de Procesamiento Distribuido	7
Batch Processing	7
Stream Processing	7
Ventajas del Procesamiento Distribuido	7
Motores de Procesamiento Distribuido	8
Miniglosario	9
Referencias	10

Conceptos de Procesamiento Distribuido

¿Qué es el Procesamiento de Datos?

Antes de ver en qué consiste el Procesamiento Distribuido, necesitamos entender qué significa el Procesamiento de Datos. Este concepto se refiere al consumo masivo de diversos datos desde diferentes fuentes como servicios, aplicaciones, archivos, bases de datos, entre otros, y a la ejecución de distintos tipos de algoritmos sobre dicho conjunto de datos con el objetivo de transformarlos en información valiosa, ya sea para los procesos de un negocio, ya sea para la investigación en determinada área. Esta obtención de información a partir de los datos es comúnmente conocida como Analítica de Datos, término que hemos visto en anteriores unidades. Gracias a la Analítica de Datos, las empresas pueden acceder a información que les permite crear mejores productos, saber qué quieren sus clientes, sus gustos y sus patrones de uso. Toda esta información ayuda a acelerar el crecimiento de la empresa en cuestión.

¿Qué es el Procesamiento Distribuido de Datos?

En el mundo de la Gestión de Grandes Volúmenes de Datos y de Big Data, existen dos términos importantes: el Almacenamiento Distribuido de Datos y el Procesamiento Distribuido de Datos. Los dos términos surgen como consecuencia del aumento exponencial en la generación, la transferencia y el almacenamiento de datos, factores causados por el auge del Internet de las Cosas. Hablar de Almacenamiento Distribuido implica hablar de sistemas distribuidos, es decir, dispositivos físicos o virtuales independientes, comúnmente denominados **nodos**, que se conectan entre sí por medio de una red de comunicaciones y que trabajan en conjunto para cumplir un mismo objetivo. Este sistema distribuido debe dar la impresión, de cara a nosotros como usuarios, de ser un único equipo, cumpliendo determinada tarea. Para que un sistema sea considerado distribuido, debe cumplir con las siguientes características:

1. Tener la posibilidad de ejecutar tareas de manera concurrente o paralela.
2. Ser transversales a las tecnologías, es decir, no deben depender ni de sistemas operativos, lenguajes de programación, etc.
3. Ser horizontalmente escalables, es decir, deben tener la capacidad de agregar nuevos nodos al sistema.

4. Ser tolerantes a fallos. Esto significa que si un nodo deja de funcionar, siempre debe existir otro que lo respalde.
5. Ser transparentes al usuario. Los usuarios deben pensar que están utilizando un único dispositivo o máquina.
6. Tener reloj interno independiente.
7. Tener memoria independiente.

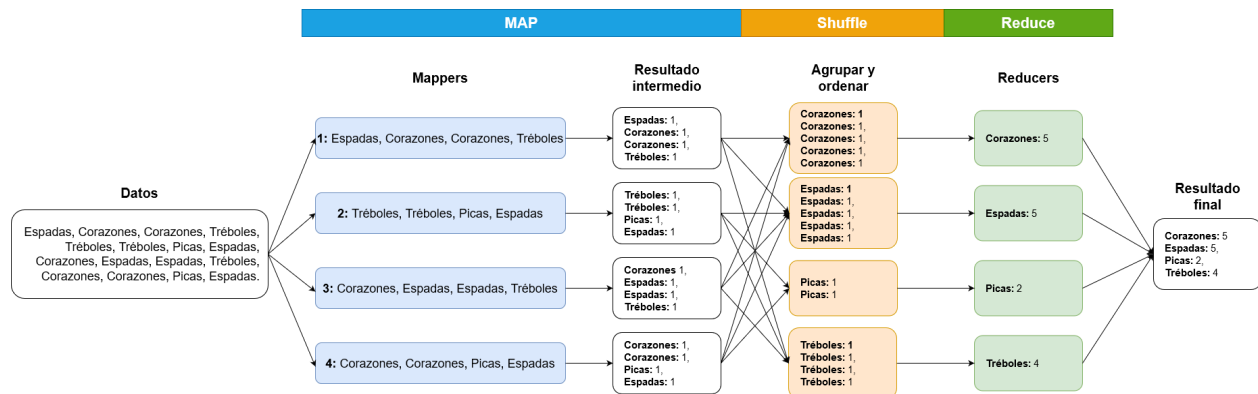
Cuando hablamos de Procesamiento Distribuido, todas estas características también aplican, esto debido a que los dos términos van muy de la mano; normalmente para poder realizar un Procesamiento Distribuido se debe contar con un Almacenamiento Distribuido. Por ejemplo, el Procesamiento Distribuido también es horizontalmente escalable, lo que le permite agregar nodos a la estructura de procesamiento si así se requiere. La principal diferencia radica en que la separación de los datos en los diferentes nodos del sistema está enfocada al procesamiento y no tanto al almacenamiento. Cabe resaltar que el Procesamiento Distribuido implica mucho más que un sistema distribuido; implica la construcción de modelos computacionales que funcionen en sistemas distribuidos y la implementación de algoritmos que resuelvan problemas de manera distribuida. Un buen ejemplo de lo que es el Procesamiento Distribuido es la **computación en la nube**.

Modelos de Procesamiento Distribuido

Como se mencionó anteriormente, el Procesamiento Distribuido implica la construcción de modelos computacionales que se ajusten a las características de las infraestructuras distribuidas. Veamos a continuación los dos modelos principales cuando de Procesamiento Distribuido de Datos se trata.

Modelo MapReduce

MapReduce es un modelo de programación que consiste en el uso de dos funciones principales para el procesamiento distribuido de los datos: **map** y **reduce**. Se utiliza para trabajar principalmente con grandes cantidades de datos, del orden de Terabytes e, incluso, de Petabytes. Este modelo apareció por primera vez en 2004, en un artículo de computación escrito por dos investigadores de la empresa Google. Algo interesante es que MapReduce pertenece al mundo del Open Source, por lo que está disponible para que cualquier persona lo utilice y lo modifique (también se puede contribuir al proyecto). En la siguiente imagen podemos observar el esquema general del modelo.



Para entender cómo funciona el modelo, imaginemos que estamos en una partida de cartas y queremos saber rápidamente cuántas cartas de cada tipo tenemos en nuestra baraja. Utilizando MapReduce, los pasos para resolver el interrogante serían:

1. Identificamos el input de datos del modelo como un texto cuyo contenido de palabras representan las cartas de nuestra baraja.
2. Dividimos el input en conjuntos de datos más pequeños, denominados **chunks**. La estructura de estos chunks corresponde a una pareja clave-valor donde la clave corresponde al número de línea y el valor a la línea de palabras en sí.
3. Distribuimos los chunks en los **mappers** del modelo. Cada mapper ejecuta una función **map** cuya lógica depende de qué desee hacer el usuario con el conjunto de datos. En nuestro ejemplo, lo que haremos será, simplemente, separar cada palabra y asignarle el número 1, mismo que corresponde a una aparición en el chunk de datos. El resultado de cada mapper es un chunk con la lista de palabras, que actúan como llaves, y un valor de 1 asignado a cada una.
4. Llegados a este punto, se realiza un proceso llamado **shuffle**, encargado de agrupar los resultados y ordenarlos. En este caso, agrupamos los diferentes resultados por palabra en nuevos chunks y los ordenamos alfabéticamente, facilitando así el trabajo de los reducers.
5. Los **reducers** se encargan de hacer el conteo de cuántas veces apareció una palabra y generan como respuesta nuevos chunks que contienen la palabra, a modo de llave, y el número contabilizado como valor final.
6. Por último, la salida del modelo es la unión de los resultados de todos los reducers.

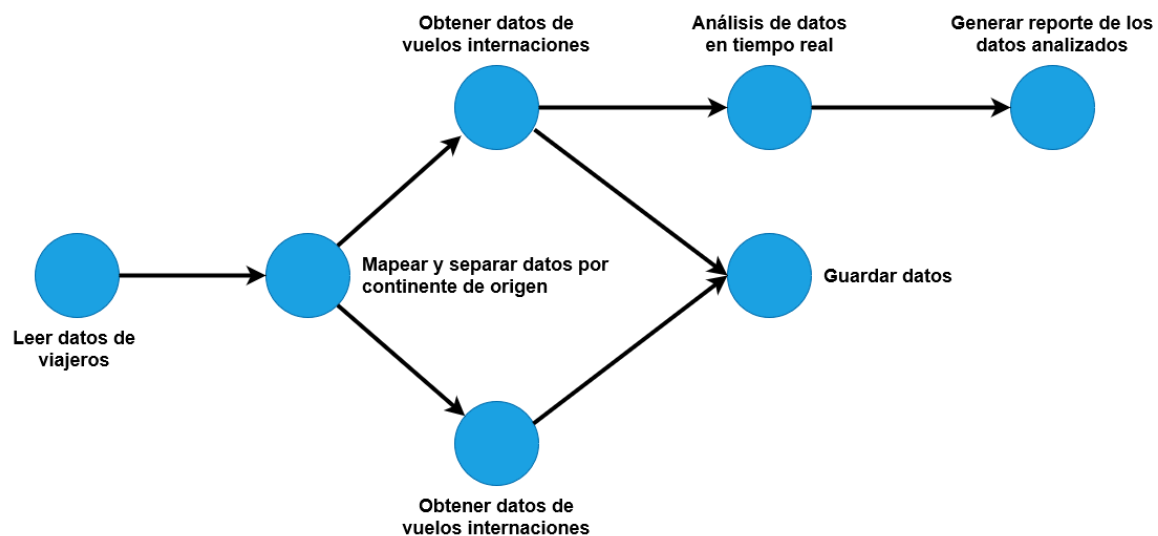
Como podemos ver, MapReduce es un conjunto de pasos bien definidos, cuya lógica se basa principalmente en el uso de pares **clave-valor** para el procesamiento de los datos. Recordemos que este modelo de programación

está enfocado, principalmente, a ejecutarse de manera distribuida en múltiples nodos del sistema, por lo que cada tarea perteneciente a los procesos **map**, **shuffle** y **reduce** se realiza de manera concurrente en diferentes nodos. Su principal innovación es que no se necesita que los nodos del sistema tengan muchos recursos de hardware, debido a que los chunks son conjuntos pequeños de datos. Los nodos pueden ser computadoras físicas o virtuales sencillas y relativamente baratas.

Este es, a grandes rasgos, el funcionamiento del modelo de programación MapReduce. En caso de querer profundizar en este modelo, la recomendación es leer el documento publicado por Google, mismo que se ofrece como insumo de la presente unidad.

Modelo Directed Acyclic Graph, DAG

Un Directed Acyclic Graph, o DAG, es un tipo especial de grafo que permite representar un conjunto de datos que se relacionan entre sí. Visualmente se muestra como un conjunto de nodos -o vértices-, conectados por líneas -o bordes-. Los vértices representan conjuntos de datos mientras que los bordes representan el flujo de datos o acciones que se realizan sobre cada conjunto. Al ser un grafo de tipo dirigido, cada vértice está unido a otro por un borde que define una dirección. Por otra parte, al ser acíclico, no se presentan ciclos, por lo que no se pueden volver a visitar vértices ni bordes por los que ya se haya pasado. En la siguiente imagen podemos ver la representación esquemática de este modelo.



En el ejemplo de la imagen observamos un DAG en el que se realizan diferentes acciones sobre los datos de viajeros. Primero se leen y cargan los

datos que serán procesados en la estructura del DAG; después el DAG ejecuta un algoritmo de mapeo sobre dichos datos, separándolos por el continente de origen, por ejemplo, América y Europa; a continuación, para cada grupo de viajeros se realiza un proceso para obtener los datos de los vuelos internacionales; posteriormente esos datos procesados se guardan en alguna fuente de datos de destino. Ahora bien, mientras los datos se están guardando, se realiza un análisis en tiempo real sobre los datos de vuelos internacionales de uno de los dos grupos para, finalmente, generar un reporte de análisis. Este es, a groso modo, el funcionamiento de un DAG, para el que las posibilidades de uso pueden llegar a ser muy variadas. Las dos características principales de los DAG, Acíclico y Dirigido, hacen de estas estructuras herramientas muy útiles para representar flujos de datos, ya que permiten organizar dichos flujos, así como las bifurcaciones originadas por las acciones sobre estos. Como consecuencia, los DAG son muy utilizados actualmente para gestionar tareas en sistemas operativos, compiladores de lenguajes de programación, procesos de manipulación de flujos de datos y optimización de código. Son usados además en el análisis de Redes Bayesianas, ayudando considerablemente en diferentes ramas científicas como la Epidemiología, la Inteligencia Artificial y la Física Cuántica. Finalmente, han mostrado un enorme potencial en las operaciones y revisiones de datos en redes de datos distribuidas. Podemos resumir las propiedades, características y funcionamiento de un DAG en los siguientes puntos:

1. Los nodos o **vértices** del DAG se encargan de almacenar los conjuntos de datos.
2. Los **bordes** o líneas son dirigidos, es decir, van en una única dirección, lo que hace que no se pueda volver a pasar por un vértice ni por un borde una segunda vez.
3. Poseen un nodo o vértice ancestral, es decir, es un vértice que no tiene antecesores o padres.
4. Poseen nodos o vértices que no tienen hijos, mismo que reciben el nombre de **hojas**.
5. De los dos puntos anteriores, se debe tener claro que su estructura es diferente a los **árboles**. En un árbol un vértice puede tener varios hijos y un único padre; en un DAG un vértice puede tener varios hijos y varios padres.
6. Al ser un elemento originado a partir de la Teoría de Grafos y tener como características principales el ser acíclico y dirigido, posee naturaleza matemática que le otorga propiedades adicionales como, por ejemplo, el **ordenamiento topológico**. Esta última propiedad le confiere la posibilidad de ejecutar procesos en un modelo de **pipeline**, donde la entrada de un proceso es el resultado de uno que lo precede.

Tipos de Procesamiento Distribuido

El procesamiento distribuido se divide en dos tipos principales: Batch Processing y Stream Processing. Veamos en qué consiste cada uno.

Batch Processing

Este tipo de procesamiento consiste en tareas automatizadas que actúan sobre datos recopilados en lapsos o periodos de tiempos determinados. Usualmente, los datos son extraídos de una base de datos, o de una fuente de datos ubicada en disco, se les aplica el algoritmo de procesamiento y el resultado se guarda en la misma base de datos o en fuentes de destino diferentes. El Modelo MapReduce se encuentra categorizado en este tipo de procesamiento. De igual forma, el modelo DAG puede ser adaptado para que funcione como un Batch Processing.

Stream Processing

El Stream Processing consiste en aplicaciones que reciben continuamente datos, los procesan y proveen los resultados en **tiempo real**. A diferencia del Batch Processing, en el Stream Processing no se debe esperar a extraer los datos cada cierto periodo de tiempo pues, mientras un nodo accede a los datos, otro los puede estar procesando. DAG es un modelo que funciona bajo el concepto de Stream Processing.

Ventajas del Procesamiento Distribuido

Gracias al Procesamiento Distribuido, las tareas asignadas se pueden realizar en un tiempo considerablemente menor, en contraste con los tiempos que toman las mismas tareas en un ambiente completamente centralizado. Esto se debe a que los equipos de cómputo comparten las tareas y las ejecutan de manera concurrente o paralela, en lugar de utilizar una **cola sincrónica**. Esta disminución en tiempos de ejecución representa, a su vez, una disminución en los costos de los procesos de negocio, a la par que un aumento en la velocidad de sus acciones. Otras ventajas son la alta escalabilidad, la alta disponibilidad y la alta tolerancia a fallos, pues se pueden agregar nodos al sistema que sirvan de respaldo para ejecutar las tareas de aquellos nodos que presenten fallos.

Motores de Procesamiento Distribuido

Los motores son herramientas que nos van a permitir implementar códigos eficientes de manera sencilla, sin tener que preocuparnos por la complejidad que implica la construcción de una infraestructura de Procesamiento Distribuido de Datos. Estos motores soportan la **ingesta** o consumo masivo de datos, el procesamiento y filtrado de los mismos y las consultas a diferentes fuentes de datos estructurados, semiestructurados y no estructurados. Además poseen la capacidad de almacenar y exportar grandes volúmenes de datos; son capaces de gestionar nodos del sistema distribuido, reiniciando una tarea en un mismo nodo o en otro diferente si llega a ocurrir una falla. En ese sentido, programan las tareas que los nodos deben realizar y pueden monitorearlas. En esta unidad veremos dos herramientas de procesamiento distribuido muy conocidas: Hadoop y Spark. A partir de este punto, la recomendación es ver los videos de la unidad donde veremos el funcionamiento de cada motor y donde realizaremos una práctica en Hadoop.

Miniglosario

Concurrencia: tareas u operaciones de un proceso que se ejecutan de forma paralela, es decir, al mismo tiempo.

Coordinación: colaboración fluida entre las operaciones y eventos que tienen lugar en determinado procesamiento.

Estado global: unión de todos los estados particulares de cada una de las tareas separadas que tienen lugar en la ejecución de determinado procesamiento. Provee una visión completa de todo el sistema.

Middleware: capa que conecta a los nodos entre sí y los hace parecer como un único nodo. Se encarga de la comunicación, la seguridad, el manejo de errores, en el entramado del sistema de Procesamiento Distribuido.

Modelo de arquitectura: modelo que condiciona la forma en la que se organizan los nodos, cómo se comunican y cómo interactúan entre ellos. El modelo provee una mejor administración y facilidad de mantenimiento al sistema y al procesamiento.

Nodo: componente de hardware -físico- o software -virtual- que posee su propio procesador y memoria, y que puede comunicarse con los demás integrantes del sistema distribuido.

Recurso: bien o activo intangible, accedido de forma remota por nodos o usuarios de las aplicaciones del sistema distribuido. Corresponde a una fuente de datos y puede ser un servicios, un archivos, una bases de datos, entre otros.

Sincronización: proceso en el que se ordenan los eventos y se controla el acceso a los recursos, de forma que se eviten inconsistencias en el procesamiento.

Transparencia: término que indica que todo el procesamiento distribuido y todos los elementos implicados en el mismo no son visibles de cara al usuario. El usuario debe pensar que todo lo hace un solo equipo o nodo.

Referencias

1. Anderson, J. *Learn MapReduce with Playing Cards*. Agosto de 2013. [En línea]: disponible en <https://www.youtube.com/watch?v=bcjSe0xCHbE>. [Accedido en 2020].
2. CBT Nuggets. *MicroNugget. What is MapReduce?* Septiembre de 2013. [En línea]: disponible en <https://www.youtube.com/watch?v=s8EPQpgpWVE>. [Accedido en 2020].
3. Kodzoman, V. *Introduction to batch processing - MapReduce*. Octubre de 2017. [En línea]: disponible en <https://datawhatnow.com/batch-processing-mapreduce/>. [Accedido en 2020].
4. Le, J. *An Introduction to Big Data: Distributed Data Processing*. 2019. [En línea]: disponible en <https://jameskle.com/writes/distributed-data-processing>. [Accedido en 2020].
5. M. C. Pabón. *Sistemas de Bases de Datos: Procesamiento Distribuido*. Extracto del curso Sistemas de Bases de Datos de la Maestría en Ingeniería de Software. Pontificia Universidad Javeriana. Facultad de Ingeniería.
6. Maldonado, J. *¿Qué es DAG y cómo funciona?* Abril de 2020. [En línea]: disponible en <https://es.cointelegraph.com/explained/what-is-a-dag-and-how-do-it-work>. [Accedido en 2020].
7. Shivang. *Distributed Data Processing 101 - The Only Guide You'll Ever Need*. [En línea]: disponible en <https://www.8bitmen.com/distributed-data-processing-101-the-only-guide-youll-ever-need/>. [Accedido en 2020].
8. That C# guy. *¿Qué es MapReduce?* Febrero de 2018. [En línea]: disponible en <https://www.youtube.com/watch?v=b27hWyhq9oo>. [Accedido en 2020].
9. The TechCave. *Distributed Systems | Distributed Computing Explained*. Diciembre de 2019. [En línea]: disponible en <https://www.youtube.com/watch?v=ajjOEltiZm4>. [Accedido en 2020].